

Models

Professor Cian Dorr

1st November 2022

New York University

First-order structures

First-order structures

Definition

A *structure* for a first-order signature Σ is a tuple $\langle D, (I_s)_{s \in R_\Sigma}, (I_t)_{t \in F_\Sigma} \rangle$, where:

- ▶ D (called the “domain” of the structure) is not empty
- ▶ $I_s \subseteq D^n$ when $s \in R_\Sigma$ and $a_\Sigma s = n$
- ▶ $I_t \in D^{D^n}$ when $t \in F_\Sigma$ and $a_\Sigma s = n$.

Example

The *standard model of arithmetic* is the structure for the language of arithmetic such that $D = \mathbb{N}$, and for all $m, n \in D$, $I_{\leq} = \{\langle n, m \rangle \mid n \leq m\}$, $I_0 = 0$, $I_{\text{suc}}(n) = \text{suc } n$, $I_+(n, m) = n + m$, $I_\cdot(n, m) = n \times m$.

NB: there are many, many other structures for this same signature!

Assignment functions

Definition

When S is a structure for Σ with domain D , an *assignment function* for S is a function from Var to D .

Occasionally we may consider *partial assignment functions* for a structure, which are functions from some $V \subseteq \text{Var}$ to the structure's domain.

The following notation will be helpful:

Definition

Suppose g is a [partial] assignment function for a structure with domain D ; v is a variable; and $d \in D$. Then $g[v \mapsto d]$ is the [partial] assignment function such that

$$g[v \mapsto d](u) = \begin{cases} d & \text{if } u = v \\ g(u) & \text{otherwise} \end{cases}$$

Why do we care about assignment functions?

We are interested in defining a notion of what it takes for a *sentence* of $\mathcal{L}(\Sigma)$ to be “true in” a structure for Σ .

But a sentence is a special case of a formula (one with no free variables), and for a formula that *does* have free variables, it doesn't seem very sensible to ask whether it's true full stop. Is the formula $x \leq y$ true in the standard model of arithmetic? We want to say, ‘It's true if you take x to stand for 0 and y to stand for 1; false if you take x to stand for 2 and y to stand for 0;...’. In other words: for open formula, the notion we want is **truth in a structure on an assignment**. And since many *sentences* have open formulae as constituents, we'll need to consider this even if all we're ultimately interested in is the truth of sentences in a structure.

To get this into the format for a recursive definition, we'll define a function $\llbracket \cdot \rrbracket_S$ that maps each formula P to a set $\llbracket P \rrbracket_S$ of assignments for S . Then we can define ‘ P is true in S on g ’ as ‘ $g \in \llbracket P \rrbracket_S$ ’. (Also pronounced ‘ g satisfies P in S ’.)

Denotations of formulae in a relational signature

For simplicity, let's start with relational signatures (no function symbols)

Definition

Where Σ is a relational signature and S is a Σ -structure with domain D , $\llbracket \cdot \rrbracket_S : \mathcal{L}(\Sigma) \rightarrow \mathcal{P}(D^{\text{Var}})$ is the function such that:

$$\begin{aligned}\llbracket F(v_1, \dots, v_n) \rrbracket &= \{g \mid \langle gv_1, \dots, gv_n \rangle \in I_F\} & \llbracket v_1 = v_2 \rrbracket &= \{g \mid gv_1 = gv_2\} \\ \llbracket \neg P \rrbracket &= \{g \in D^{\text{Var}} \mid g \notin \llbracket P \rrbracket\} & \llbracket P \wedge Q \rrbracket &= \llbracket P \rrbracket \cap \llbracket Q \rrbracket \\ \llbracket P \rightarrow Q \rrbracket &= \{g \in D^{\text{Var}} \mid g \notin \llbracket P \rrbracket \text{ or } g \in \llbracket Q \rrbracket\} & \llbracket P \vee Q \rrbracket &= \llbracket P \rrbracket \cup \llbracket Q \rrbracket \\ \llbracket \forall v P \rrbracket &= \{g \mid g[v \mapsto d] \in \llbracket P \rrbracket \text{ for all } d \in D\} \\ \llbracket \exists v P \rrbracket &= \{g \mid g[v \mapsto d] \in \llbracket P \rrbracket \text{ for some } d \in D\}\end{aligned}$$

(Here I left the subscript S implicit to save space.)

The same thing put in more familiar terms

In other words:

- ▶ $F(v_1, \dots, v_n)$ is true in S on g iff $\langle gv_1, \dots, gv_n \rangle \in I_F$.
- ▶ $v_1 = v_2$ is true in S on g iff $gv_1 = gv_2$.
- ▶ $\neg P$ is true in S on g iff P isn't true in S on g .
- ▶ $P \rightarrow Q$ is true in S on g iff P isn't true in S on g or Q is true in S on g .
- ▶ $P \wedge Q$ is true in S on g iff P and Q are both true in S on g .
- ▶ $P \vee Q$ is true in S on g iff either P or Q is true in S on g .
- ▶ $\forall v P$ is true in S on g iff P is true in S on $g[v \mapsto d]$ for all d in the domain.
- ▶ $\exists v P$ is true in S on g iff P is true in S on $g[v \mapsto d]$ for some d in the domain.

Examples

Consider the structure N for the signature with just one 2-place predicate $\leq$, with domain $\mathbb{N}$, where $I_{\leq}mn$ iff $m \leq n$.

- ▶ $g \in \llbracket x \leq y \rrbracket_N$ iff $gx \leq gy$
- ▶ $g \in \llbracket \forall y(x \leq y) \rrbracket_N$ iff for all $n \in \mathbb{N}$, $g[y \mapsto n] \in \llbracket x \leq y \rrbracket_N$
iff for all $n \in \mathbb{N}$, $g[y \mapsto n](x) \leq g[y \mapsto n](y)$
iff for all $n \in \mathbb{N}$, $gx \leq n$
- ▶ $g \in \llbracket \exists x \forall y(x \leq y) \rrbracket_N$ iff for some $m \in \mathbb{N}$, $g[x \mapsto m] \in \llbracket \forall x(x \leq y) \rrbracket_N$
iff for some $m \in \mathbb{N}$: $g[x \mapsto m](x) \leq n$ for all $n \in \mathbb{N}$
iff for some $m \in \mathbb{N}$: $m \leq n$ for all $n \in \mathbb{N}$
iff g is an assignment for N .

By contrast, the same sentence is true on *no* assignments in the structure where $I_{\leq}mn$ is true iff $m < n$.

Only free variables make a difference

Irrelevance Lemma

If $gv = hv$ for all $v \in FV(P)$, then $h \in \llbracket P \rrbracket_S$ if $g \in \llbracket P \rrbracket_S$.

Proof: by induction on the construction of P . We just do a few clauses.

Base case: suppose P is an atomic formula $F(v_1, \dots, v_n)$ true on g , and h agrees with g on $FV(P) = \{v_1, \dots, v_n\}$. Then $\langle gv_1, \dots, gv_n \rangle = \langle hv_1, \dots, hv_n \rangle$. Since $\langle gv_1, \dots, gv_n \rangle \in I_F$, $\langle hv_1, \dots, hv_n \rangle \in I_F$, so P is true on h .

Conjunction: Suppose that P and Q are each such that whether they are true on an assignment depends only on what it sends their free variables to. Suppose $P \wedge Q$ is true on g , and g and h agree on $FV(P \wedge Q) = FV(P) \cup FV(Q)$. Then they agree on $FV(P)$ and on $FV(Q)$, so P and Q are both true on h by the IH. So by definition of $\llbracket P \wedge Q \rrbracket$, $P \wedge Q$ is true on h .

Universal quantification: Suppose that whenever g and h agree on $FV(P)$, P is true on g iff it's true on h . Now suppose $\forall v P$ is true on g , and h agrees with g on $FV(\forall v P) = FV(P) \setminus \{v\}$. For any d in the domain, $h[v \mapsto d]$ agrees with $g[v \mapsto d]$ on $FV(P)$. For any given such d , by definition of $\llbracket \forall v P \rrbracket$, P is true on $g[v \mapsto d]$; so by the IH, P is true on $h[v \mapsto d]$. Hence $\forall v P$ is true on h .

The other steps are similar.

Truth in a structure for sentences

As a special case, if P is a *sentence*, i.e. $FV(P) = \emptyset$, then $\llbracket P \rrbracket_S$ either contains every assignment for S or contains no assignments for S .

Definition

Suppose P is a sentence of $\mathcal{L}(\Sigma)$ and S is a structure for Σ . Then P is **true in S** iff it is true in S on every assignment.

Given what we just established, we could just as well have written ‘on some assignment’.

Adding function symbols

Up to now we have been ignoring function symbols, for simplicity. To add them back, we will need a subsidiary recursive definition applicable to the *terms* of the signature. We use the same notation $\llbracket \cdot \rrbracket_S$ for this, but it's a different sort of thing: for a term t , $\llbracket t \rrbracket_S$ is not a *set* of assignment functions, but a *function* from the set of all assignment functions to the domain of S .

We call $\llbracket t \rrbracket_S g$ the **denotation of t on g** . We write it as $\llbracket t \rrbracket_S^g$, or just $\llbracket t \rrbracket^g$ when it's obvious what structure we're talking about.

Definition

Given a structure S for Σ , $\llbracket \cdot \rrbracket_S$ is the function from $\text{Terms}(\Sigma)$ to $D^{D^{\text{Var}}}$ such that

1. $\llbracket v \rrbracket_S^g = gv$ for any variable v .
2. $\llbracket c \rrbracket_S^g = I_c$ if c is an individual constant of Σ (for any g).
3. $\llbracket f(t_1, \dots, t_n) \rrbracket_S^g = I_f(\llbracket t_1 \rrbracket_S^g, \dots, \llbracket t_n \rrbracket_S^g)$

Examples of denotations of terms relative to assignment

Generalizing the definition of denotation for formulae

This lets us extend our previous definition to non-relational signatures:

Definition

Where Σ is a relational signature and S is a Σ -structure with domain D ,
 $\llbracket \cdot \rrbracket : \mathcal{L}(\Sigma) \rightarrow \mathcal{P}(D^{\text{Var}})$ such that:

$$\begin{aligned}\llbracket F(v_1, \dots, v_n) \rrbracket &= \{g \mid \langle gv_1, \dots, gv_n \rangle \in I_F\} & \llbracket v_1 = v_2 \rrbracket &= \{g \mid gv_1 = gv_2\} \\ \llbracket \neg P \rrbracket &= \{g \in D^{\text{Var}} \mid g \notin \llbracket P \rrbracket\} & \llbracket P \wedge Q \rrbracket &= \llbracket P \rrbracket \cap \llbracket Q \rrbracket \\ \llbracket P \rightarrow Q \rrbracket &= \{g \in D^{\text{Var}} \mid g \notin \llbracket P \rrbracket \text{ or } g \in \llbracket Q \rrbracket\} & \llbracket P \vee Q \rrbracket &= \llbracket P \rrbracket \cup \llbracket Q \rrbracket \\ \llbracket \forall v P \rrbracket &= \{g \mid g[v \mapsto d] \in \llbracket P \rrbracket \text{ for all } d \in D\} \\ \llbracket \exists v P \rrbracket &= \{g \mid g[v \mapsto d] \in \llbracket P \rrbracket \text{ for some } d \in D\}\end{aligned}$$

Generalizing the definition of denotation for formulae

This lets us extend our previous definition to non-relational signatures:

Definition

Where Σ is a relational signature and S is a Σ -structure with domain D ,
 $\llbracket \cdot \rrbracket : \mathcal{L}(\Sigma) \rightarrow \mathcal{P}(D^{\text{Var}})$ such that:

$$\llbracket F(t_1, \dots, t_n) \rrbracket = \{g \mid \langle \llbracket t_1 \rrbracket^g, \dots, \llbracket t_n \rrbracket^g \rangle \in I_F\}$$

$$\llbracket t_1 = t_2 \rrbracket = \{g \mid \llbracket t_1 \rrbracket^g = \llbracket t_2 \rrbracket^g\}$$

$$\llbracket \neg P \rrbracket = \{g \in D^{\text{Var}} \mid g \notin \llbracket P \rrbracket\}$$

$$\llbracket P \wedge Q \rrbracket = \llbracket P \rrbracket \cap \llbracket Q \rrbracket$$

$$\llbracket P \rightarrow Q \rrbracket = \{g \in D^{\text{Var}} \mid g \notin \llbracket P \rrbracket \text{ or } g \in \llbracket Q \rrbracket\}$$

$$\llbracket P \vee Q \rrbracket = \llbracket P \rrbracket \cup \llbracket Q \rrbracket$$

$$\llbracket \forall v P \rrbracket = \{g \mid g[v \mapsto d] \in \llbracket P \rrbracket \text{ for all } d \in D\}$$

$$\llbracket \exists v P \rrbracket = \{g \mid g[v \mapsto d] \in \llbracket P \rrbracket \text{ for some } d \in D\}$$

Example

In a structure S for the language of arithmetic,

$$g \in \llbracket \forall x (x = x + 0) \rrbracket$$

$$\text{iff for all } d, g[x \mapsto d] \in \llbracket x = x + 0 \rrbracket$$

$$\text{iff for all } d, \llbracket x \rrbracket(g[x \mapsto d]) = \llbracket x + 0 \rrbracket(g[x \mapsto d])$$

$$\text{iff for all } d, d = I_+(d, \llbracket 0 \rrbracket(g[x \mapsto d]))$$

$$\text{iff for all } d, d = I_+(d, I_0)$$

If S is the standard model of arithmetic this holds for all g ; in other structures, it doesn't.

A little more notation

For a formula P , structure S , and assignment g for S , we define

$$\llbracket P \rrbracket_S^g = \begin{cases} 1 & \text{if } g \in \llbracket P \rrbracket_S \\ 0 & \text{otherwise.} \end{cases}$$

On this picture we think of 1 and 0 as “truth values”, and think of formulae as “denoting” truth values relative to assignments, just as terms denote elements of the domain relative to assignments.

- This notation would fit together more neatly if we had defined $\llbracket P \rrbracket_S$ to be a *function from assignment functions to $\{0, 1\}$* , rather than a set of assignment functions. But these are tantamount to the same thing—in general, we can go back and forth between subsets of some set and the corresponding characteristic functions (functions from the big set to $\{0, 1\}$).

Defining logical notions

Defining logical notions

In a great marketing coup, the founders of model theory adopted a bunch of technical uses of ordinary words that make it sound like the study of models should be the central thing in logic. For example:

Definition

Formula P of $\mathcal{L}(\Sigma)$ is **valid** iff P is true in every Σ -structure on every assignment.

This generalizes to sequents $\Gamma \triangleright P (= \langle \Gamma, P \rangle)$

Definition

Sequent $\Gamma \triangleright P$ of $\mathcal{L}(\Sigma)$ is **valid** iff for every Σ -structure S and every assignment g for S , if every member of Γ is true in S on g , then P is true in S on g .

Another way of saying that the sequent $\Gamma \triangleright P$ is valid is to say that P is a **logical consequence** of Γ , or in symbols, $\Gamma \models P$.

Also,

Definition

Formula P of $\mathcal{L}(\Sigma)$ is **satisfiable** (a.k.a. logically consistent) iff there exists a Σ -structure S and an assignment g for S such that P is true in S on g .

And more generally

Definition

A set Γ of formulae of $\mathcal{L}(\Sigma)$ is **satisfiable** (logically consistent) iff there exists a Σ -structure S and an assignment g for S such that every member of Γ is true in S on g .

Relating validity and satisfiability

Fact

P is valid iff $\neg P$ is not satisfiable; P is satisfiable iff $\neg P$ is not valid.

Proof: P is true in S on g iff $\neg P$ is not true in S on g , so P is true in every structure on every assignment iff $\neg P$ isn't true on any structure on any assignment. More generally, and by the same reasoning:

Fact

$\Gamma \models P$ iff $\Gamma \cup \{\neg P\}$ is not satisfiable; Γ is satisfiable iff $\Gamma \not\models \perp$.

(Here, $\perp$ is shorthand for the formula $\neg\forall x(x = x)$).

The Soundness Theorem

The Soundness Theorem

One very central reason for being interested in models comes from the following key theorem:

Soundness Theorem

Whenever $\Gamma \vdash P$, $\Gamma \models P$.

Up to now we have had great ways of showing that sequents are provable (e.g. constructing a proof of them), but no good ways of showing that they *aren't* provable. We can look at a bunch of attempts and say 'This isn't a proof, and this isn't, and this isn't. . .'; but this procedure never rules out that there's a proof we haven't considered yet.

Thanks to the Soundness Theorem, we have a way of showing that $\Gamma \not\vdash P$: we construct a structure S and assignment g such that every member of Γ is true in S on g , but P isn't true in S on g .

More on the philosophical significance of the Soundness Theorem

Some discussions suggest the Soundness Theorem is supposed to actually provide a justification for reasoning in accordance with the classical rules baked into $\vdash$. But this is a problematic thought for at least two separate reasons.

1. Like every proof we've ever done, the proof of the Soundness Theorem requires using many of these very rules in the metalanguage. If one were really worried about those rules, one wouldn't accept the proof.
2. The imagined justification would require, e.g. going from 'Sentence P is valid' to actually asserting a certain sentence P —accepting it as *really* true. But since there isn't a set that contains everything, it is obscure how such a transition would be justified.

Proving the Soundness Theorem

We're trying to show that *every provable sequent is valid* so of course what's needed is an *induction on provable sequents*. I'll do some cases and leave others as homework.

Assumption: Trivially $P \models P$: this just means for all S, g , if P is true in S on g , P is true in S on g .

Weakening: Suppose $\Gamma \models P$ and Δ is a set of formulae. We need to show $\Gamma, \Delta \vdash P$: in other words, for every structure S and assignment g for S , if every member of $\Gamma \cup \Delta$ is true in S on g , P is true in S on g . But if every member of $\Gamma \cup \Delta$ is true in S on g , every member of Γ is; so by our induction hypothesis, P is.

∀Intro1: Suppose $\Gamma \models P$, and suppose every member of Γ is true in S on g . Then by the induction hypothesis, P is true in S on g , so either P is true in S on g or Q is, so by the definition of $\llbracket P \vee Q \rrbracket$, $P \vee Q$ is true in S on g .

∀Elim: Suppose $\Gamma \models P \vee Q$, $\Gamma, P \models R$, and $\Gamma, Q \models R$. Suppose every member of Γ is true in S on g . Then by the IH, $P \vee Q$ is true in S on g , so by the definition of $\llbracket P \vee Q \rrbracket$, either P is true in S on g or Q is. In the former case, R is true in S on g by the second part of the IH; in the latter case, R is also true in S by the third part of the IH. So, R is true in S on g .

∀Intro: Suppose $\Gamma \models P$ and v is a variable that isn't free in any member of Γ . Suppose every member of Γ is true in a certain S on a certain g . Let d be an arbitrary member of the domain of S . Since v isn't free in Γ , $g[v \mapsto d]$ agrees with g on $FV(\Gamma)$, so by the Irrelevance Lemma, every member of Γ is true in S on $g[v \mapsto d]$, so by the IH, P is true in S on $g[v \mapsto d]$. Since this holds for every d , we can conclude (looking at the definition of $\llbracket \forall v P \rrbracket$) that $\forall v P$ is true in S on g .

The Completeness Theorem

The proof of the Soundness Theorem is straightforward and unsurprising. Much more interesting is the proof of it's companion, which we'll discuss next week:

The Completeness Theorem

If $\Gamma \models P$, then $\Gamma \vdash P$.